CHAPTER 3

Test-Driven Rails

In the previous chapter you created some basic functionality for a project-management application using test-driven development. The title of this book, though, is *Rails 5 Test Prescriptions*, not *Generic Test Prescriptions*. (As with most generics, if that book did exist, it'd probably be cheaper but with less-interesting packaging.)

In this chapter you'll augment your model testing by testing the logic of the entire Rails stack from request to response using *end-to-end* tests. To do this, you'll use a tool called Capybara to help write your end-to-end tests.

A good test suite consists of a few end-to-end tests, a lot of tests that target a single unit, and relatively few tests that cover an intermediate amount of code. By moving logic outside the Rails controller and views, you can turn those harder-to-test parts of Rails tests into code that can be tested with faster and more robust unit tests.

Let's Write Some Rails

To start a test-driven development process, it's important to have some requirements in mind. Without some sense of what your code should be doing, it's hard to write tests to describe behavior.

Requirements-gathering could be an entire book by itself (specifically, this one: *Software Requirements, 2nd Edition [Wie03]*). For purposes of this example, let's assume you're your own client and you're working on a small project, so you don't exactly need military-grade precision. Here's my informal list of the first few things you'll tackle:

- A user can enter a task, associate it with a project, and also see it on the project page.